

Technical Analysis — Real-Time Hand Gesture Recognition

Abstract

This report presents a comparative study between a classical computer vision pipeline (MediaPipe hand landmarks + geometric feature engineering + SVM) and a deep learning pipeline (YOLOv8n hand detection + ResNet18 gesture classification) for real-time recognition of six hand gestures: `like`, `dislike`, `ok`, `palm`, `fist`, `peace`. The system is evaluated on the HaGRID 30k subset, achieving **91.65% validation accuracy** with the classical approach and **99.43%** with the deep learning classifier. Both pipelines are integrated into a dual-mode real-time webcam application.

1. Methodology

1.1 Problem Formulation

Hand gesture recognition is formulated as a two-stage problem: 1. **Detection / Localization**: Identify the hand region in the image. 2. **Classification**: Assign a gesture label to the detected hand.

The classical pipeline merges detection and classification via MediaPipe's pre-trained hand landmark detector, while the deep learning pipeline explicitly separates hand detection (YOLOv8n) from gesture classification (ResNet18).

1.2 Evaluation Metrics

Metric	Description
Accuracy	Proportion of correctly classified gestures
Precision (macro)	Mean per-class precision
Recall (macro)	Mean per-class recall
F1 Score (macro)	Harmonic mean of precision and recall
mAP@0.5	Mean Average Precision for hand detection at IoU ≥ 0.5
FPS	Frames per second during inference
Latency	End-to-end inference time per frame

2. Dataset

2.1 HaGRID Subset

The [HaGRID](#) (Hand Gesture Recognition Image Dataset) is a large-scale dataset of hand gestures. For this project, a subset of ~30k images at 384p resolution was used, filtered to six classes:

Class	Description	Samples (approx.)
like	Thumb up	~5,000
dislike	Thumb down	~5,000
ok	OK sign	~5,000
palm	Open palm	~5,000
fist	Closed fist	~5,000
peace	V-sign	~5,000

2.2 Data Splits

Split	Proportion	Purpose
Train	70%	Model training
Validation	15%	Hyperparameter tuning, early stopping
Test	15%	Final evaluation

2.3 Preprocessing Pipelines

Three derived datasets were created:

1. **YOLO Dataset:** Original images with bounding box annotations converted to YOLO format (`class x_center y_center width height` , normalized).
 2. **Classifier Dataset:** Hand crops extracted using ground-truth bounding boxes, resized to 224×224.
 3. **Keypoint Dataset:** MediaPipe 21 landmarks extracted from each image, converted to 60+ geometric features (distances, angles, ratios).
-

3. Classical Pipeline

3.1 Architecture

Image → MediaPipe Hands → 21 Landmarks → Feature Extractor → SVM (RBF) →

3.2 Hand Detection — MediaPipe

- **Model:** MediaPipe Hand Landmarker (Tasks API, `hand_landmarker.task`)
- **Max hands:** 2
- **Min detection confidence:** 0.7
- **Output:** 21 normalized (x, y, z) landmarks per hand

3.3 Feature Extraction

The feature extractor computes a 60+ dimensional vector per hand:

Feature Category	Count	Description
Wrist distances	20	Euclidean distance from wrist to each joint
Finger lengths	16	Consecutive joint distances per finger
PIP angles	4	Angle at proximal interphalangeal joints
Tip-to-tip distances	10	Pairwise distances between fingertips
Finger ratios	5	Tip-to-base distance / total finger length

All distance features are normalized by palm size (wrist to middle finger MCP) for scale invariance.

3.4 Classification — SVM

- **Model:** `scikit-learn SVC(kernel='rbf', C=1.0, probability=True)`
- **Preprocessing:** `StandardScaler` (zero mean, unit variance)
- **Training:** 6,331 samples
- **Validation:** 1,342 samples

3.5 Results

Metric	Value
Validation Accuracy	91.65%
Precision (macro)	0.93
Recall (macro)	0.92
F1 (macro)	0.92
Inference time	~5–10 ms/frame (CPU)

4. Deep Learning Pipeline

4.1 Architecture

Image → YOLOv8n → Hand BBox → Crop → ResNet18 → Gesture Class

4.2 Hand Detection — YOLOv8n

- **Base model:** Ultralytics YOLOv8n (nano)
- **Task:** Single-class hand detection
- **Input size:** 640×640
- **Training:** Fine-tuned on HaGRID subset for 2 epochs (mAP@0.5 $\approx$ 0.85)
- **Device:** Apple MPS (Metal Performance Shaders)

Note: Due to training time constraints, the YOLO model was only trained for 2 epochs. It is functional but may miss small or occluded hands. For production deployment, 10–20 epochs are recommended.

4.3 Gesture Classification — ResNet18

- **Base model:** torchvision resnet18 (ImageNet pretrained)
- **Modification:** Final FC layer replaced with `nn.Linear(512, 6)`
- **Input size:** 224×224
- **Data augmentation:** Random horizontal flip, random rotation ($\pm 15^\circ$)
- **Normalization:** ImageNet mean/std
- **Optimizer:** Adam
- **Loss:** Cross-entropy
- **Device:** Apple MPS
- **Training:** 20 epochs with early stopping (best at epoch 10)

4.4 Results

Metric	Value
CNN Validation Accuracy	99.43%
CNN Validation Loss	0.0214
Best Epoch	10 / 20
Training time	~20 min (MPS)
Inference time	~10–15 ms/frame (MPS)

5. Comparative Analysis

5.1 Accuracy vs Complexity

Aspect	Classical (MediaPipe + SVM)	Deep Learning (YOLO + CNN)
Val Accuracy	91.65%	99.43%
Model size	~1 MB (SVM + scaler)	~23 MB (YOLO) + 45 MB (CNN)
Training data	6,331 keypoint vectors	7,387 cropped images
Training time	~2 min (CPU)	~20 min (MPS)
Inference	~5–10 ms (CPU)	~15–30 ms (MPS)
Explainability	High (geometric features)	Low (black-box CNN)
Dependencies	scikit-learn	PyTorch, Ultralytics

5.2 Key Observations

1. **The CNN significantly outperforms the SVM** (+7.8 percentage points), demonstrating the power of learned representations over hand-engineered features.

2. **The classical pipeline is extremely lightweight** and runs efficiently on CPU without requiring a GPU.
 3. **The deep pipeline's bottleneck is YOLO detection**, not CNN classification. With only 2 training epochs, YOLO occasionally misses hands, causing false negatives in the end-to-end pipeline.
 4. **MediaPipe is remarkably robust** to lighting, scale, and rotation variations, making the classical pipeline surprisingly competitive despite its simplicity.
-

6. Failure Analysis

6.1 Classical Pipeline Failures

Failure Mode	Cause	Mitigation
Misclassification between <code>like</code> and <code>palm</code>	Similar landmark configurations when thumb is extended	Add thumb angle features
Occluded fingers	MediaPipe may predict incorrect landmarks for hidden joints	Confidence thresholding
Multiple hands	SVM classifies each independently without context	Add spatial relationship features

6.2 Deep Learning Pipeline Failures

Failure Mode	Cause	Mitigation
Missed detections (false negatives)	YOLO trained for only 2 epochs	Train YOLO for 10–20 epochs
Small hand detection	Input resize to 640×640 loses fine details	Use higher resolution or YOLOv8s
Crop boundary artifacts	Hand partially outside bbox	Add padding to YOLO crops
Overconfidence on wrong class	CNN overfits to background textures	Stronger augmentation, dropout

7. Ethical Considerations

7.1 Privacy

The webcam pipeline processes all data locally. No images, landmarks, or predictions are transmitted to external servers. Users must provide explicit consent before camera access is enabled.

7.2 Bias and Fairness

The HaGRID dataset primarily consists of subjects from specific geographic and demographic groups (predominantly Eastern European). Performance may degrade on:

- Underrepresented skin tones - Different hand sizes (children, elderly) - Cultural gesture variations not present in the training data

This limitation must be disclosed in any production deployment.

7.3 Environmental Impact

This project uses pre-trained models (ResNet18, YOLOv8n) to avoid energy-intensive training from scratch. Fine-tuning is limited to 20 epochs. The total estimated carbon footprint is minimal compared to training large models from scratch.

7.4 Regulatory Compliance

Real-time gesture recognition in public or shared spaces may require consent under: - GDPR (EU) - CCPA (California) - Local privacy laws

8. Conclusion

This project demonstrates a complete end-to-end hand gesture recognition system with two complementary approaches:

- The **classical pipeline** offers a lightweight, explainable, and CPU-friendly solution at **91.65% accuracy**.
- The **deep learning pipeline** achieves state-of-the-art **99.43% accuracy** but requires GPU acceleration and larger model weights.

For real-time applications on resource-constrained devices, the classical pipeline is preferred. For accuracy-critical applications with GPU availability, the deep learning pipeline is superior.

Future Work

1. Train YOLOv8n for 10–20 epochs to improve detection robustness.
 2. Implement temporal smoothing (e.g., majority voting over 5 frames) for stable webcam predictions.
 3. Evaluate on a more diverse, multi-ethnic dataset to assess and mitigate bias.
 4. Quantize the CNN model (INT8) for edge deployment.
 5. Add a "no gesture" class to reduce false positives in free-running mode.
-

References

1. Kapitanov et al. "HaGRID — HAnd Gesture Recognition Image Dataset." <https://github.com/hukenovs/hagrid>
2. Google MediaPipe. "Hand Landmarker." https://developers.google.com/mediapipe/solutions/vision/hand_landmarker
3. Ultralytics. "YOLOv8." <https://github.com/ultralytics/ultralytics>
4. He et al. "Deep Residual Learning for Image Recognition." CVPR 2016.
5. scikit-learn. "Support Vector Machines." <https://scikit-learn.org/stable/modules/svm.html>